

# AgroCalc Web

Sistema Web de Gestão e Cálculo para o Agronegócio

## Relatório Técnico — Projeto Integrador

Curso: Tecnologia da Informação

Disciplinas: Linguagem de Apresentação e Estruturação de Conteúdos |

Introdução à Tecnologia da Computação | Matemática Aplicada

Professores: Silvério Luiz de Sousa | Matheus Couto | Patrícia Freitas

Paranavaí — 2025

## Resumo

---

O AgroCalc Web é um sistema web desenvolvido como Projeto Integrador, reunindo em um único produto digital os conteúdos das três unidades curriculares do semestre. O sistema é composto por três páginas HTML semânticas e responsivas, quatro scripts Python documentados com cálculos de Matemática Aplicada e um módulo de Computação com conversor de sistemas de numeração, simulador de porta lógica AND e linha do tempo da computação no agronegócio. O front-end adota variáveis CSS, CSS Grid, Flexbox, animações por keyframes e dark mode com persistência em localStorage. A lógica Python cobre função afim, ponto de equilíbrio, financiamento por juros compostos (PMT), operações matriciais com NumPy e resolução de sistema linear  $3 \times 3$  por `numpy.linalg.solve` com verificação de determinante. Um servidor Flask complementar gera gráficos Matplotlib sob demanda e os exibe no navegador, constituindo o bônus de integração Python-web.

# 1. Introdução

---

O agronegócio brasileiro responde por parcela significativa do Produto Interno Bruto nacional e depende, cada vez mais, de ferramentas digitais para otimizar custos, planejar financiamentos e monitorar produtividade. Pequenos e médios produtores rurais, no entanto, frequentemente não têm acesso a sistemas especializados — seja pelo custo das licenças, seja pela complexidade de uso.

O AgroCalc Web nasce desse cenário como resposta prática: um sistema leve, acessível via navegador, sem necessidade de instalação, que reúne as ferramentas mais recorrentes no dia a dia de uma propriedade rural. O projeto foi desenvolvido individualmente ao longo de quatro semanas, simulando um contrato freelance real em que o desenvolvedor precisa dominar simultaneamente interface, lógica de programação e matemática.

Do ponto de vista técnico, o sistema integra três camadas: (1) front-end em HTML5 e CSS3 com JavaScript puro para interatividade no navegador; (2) scripts Python independentes que implementam os cálculos matemáticos e podem ser executados via terminal; e (3) um servidor Flask que conecta os dois mundos, gerando gráficos Matplotlib dinamicamente e servindo-os como imagens para o site.

O objetivo geral é demonstrar domínio técnico nas três unidades curriculares do semestre — Linguagem de Apresentação e Estruturação de Conteúdos, Introdução à Tecnologia da Computação e Matemática Aplicada — em um produto funcional, documentado e apresentável.

## 2. Desenvolvimento — HTML e CSS

---

### 2.1 Estrutura de Arquivos

O projeto adota separação clara de responsabilidades:

- src/html/ — três páginas HTML5 (index.html, calculadora.html, computacao.html)
- src/css/styles.css — única folha de estilo externa com 1.008 linhas
- src/js/script.js — script JavaScript com 519 linhas, carregado com defer

Nenhum estilo inline foi utilizado em nenhuma das páginas, conforme exigido pelo projeto. Toda a estilização parte de variáveis CSS definidas em :root e sobrescritas para o tema claro.

### 2.2 HTML Semântico

As três páginas seguem a mesma estrutura semântica base, garantindo consistência e acessibilidade:

```
<header class="site-header">      <!-- cabeçalho com navbar -->
  <nav class="navbar" aria-label="Navegação principal">
    <!-- logo, toggle mobile, links, botão de tema -->
  </nav>
</header>
<main>
  <section class="hero">           <!-- seção de destaque -->
  <section class="section-block"> <!-- seções de conteúdo -->
</main>
<footer class="site-footer">      <!-- rodapé -->
```

Formulários utilizam elementos semânticos com atributos de acessibilidade: aria-label, aria-live nos result-box e required em todos os campos obrigatórios. O menu mobile é implementado com checkbox + label (técnica CSS pura, sem JavaScript para o toggle).

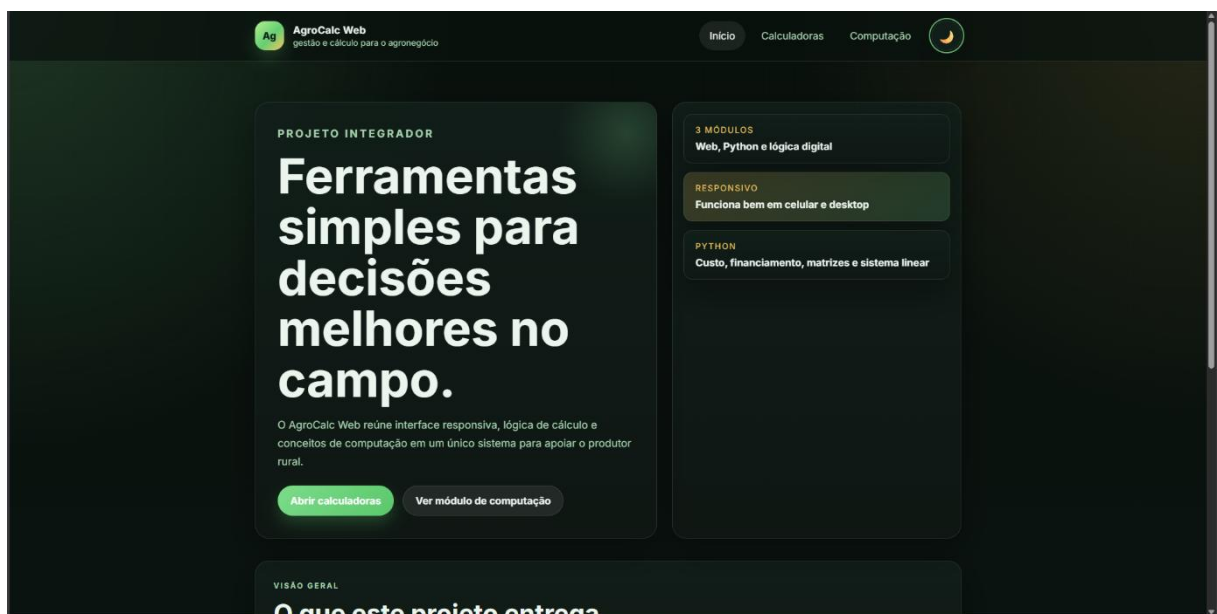
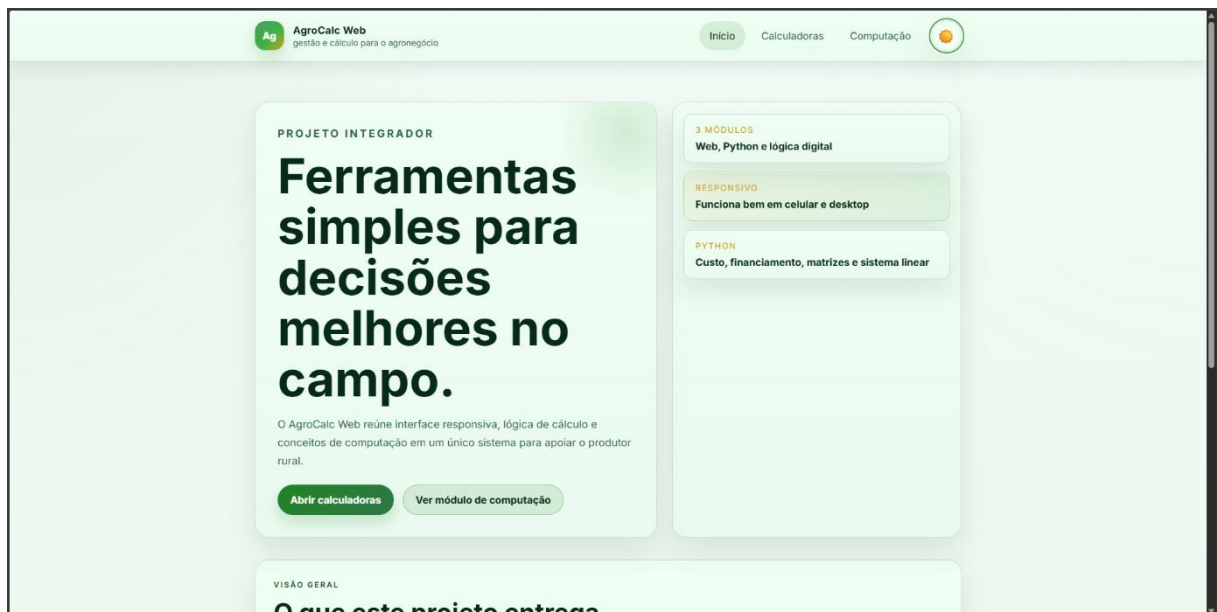
### 2.3 Variáveis CSS e Sistema de Temas

O arquivo styles.css define um sistema de design tokens completo via variáveis CSS:

```
:root {
  /* Cores base — tema dark (padrão) */
  --cor-fundo: #0c1511;
  --cor-primaria: #7ddc8b;
  --cor-secundaria: #f6c85f;
  --sombra: 0 20px 50px rgba(0,0,0,0.35);
  --raio: 22px;
  --fonte: 'Inter', system-ui, sans-serif;
}

html[data-theme="light"] {
  --cor-fundo: #eaf7ec;          /* sobrescreve apenas no light */
  --cor-primaria: #238225;
}
```

O toggle de tema é gerenciado pelo JavaScript (função `toggleTheme`) que altera o atributo `data-theme` no elemento html e persiste a escolha no `localStorage`. O modo dark é o padrão; o modo claro é ativado pelo usuário e lembrado entre sessões.



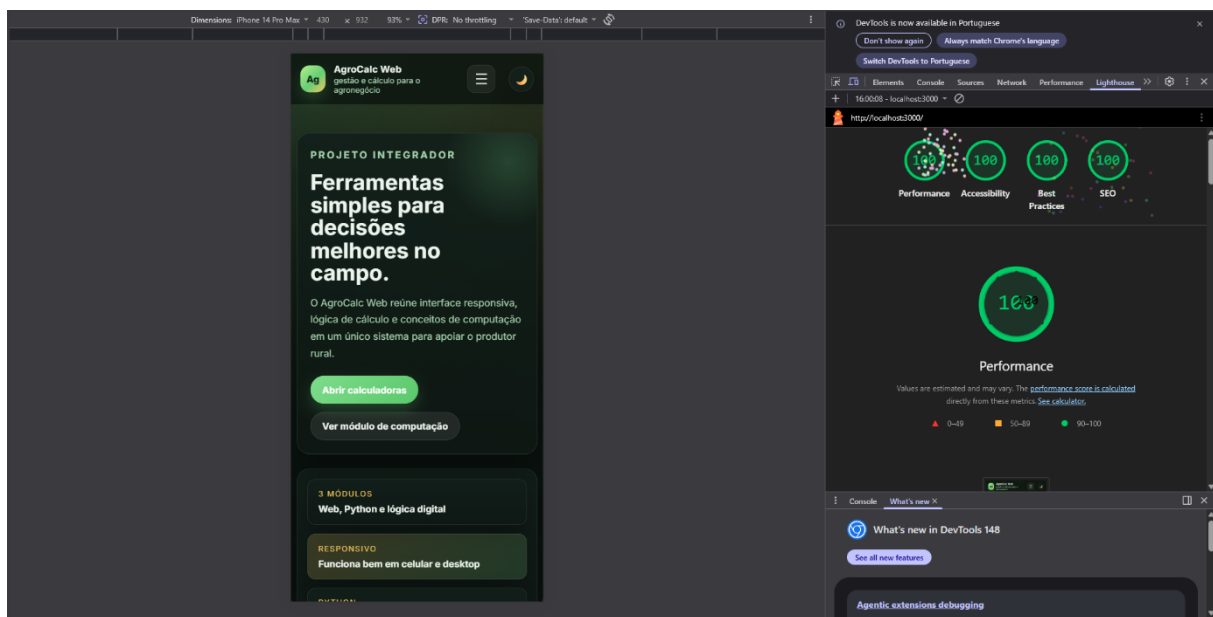
## 2.4 Layout Responsivo — CSS Grid e Flexbox

O grid de cards usa auto-fit com minmax para adaptar colunas automaticamente à largura da tela:

```
.cards-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
  gap: 1.5rem;
}
```

A navbar usa Flexbox com justify-content: space-between para distribuir logo, links e botão. Em telas menores que 860px os cards colapsam para uma única coluna e o menu hambúrguer substitui os links de navegação:

```
@media (max-width: 860px) {
  .hero, .highlight-band, .single-column-mobile {
    grid-template-columns: 1fr;
  }
}
@media (max-width: 480px) {
  /* ajustes adicionais para telas muito pequenas */
}
```



## 2.5 Animações CSS

Três animações por keyframes foram implementadas:

| Animação | Aplicada em       | Efeito                                                      |
|----------|-------------------|-------------------------------------------------------------|
| fadeUp   | Cards ao carregar | Surge de baixo com fade-in (opacity 0→1, translateY 24px→0) |
| slideIn  | Itens de lista    | Desliza da esquerda com fade-in                             |
| pulse    | Hover em botões   | Leve escala (1→1.02→1) ao passar o mouse                    |

```
@keyframes fadeUp {
  from { opacity: 0; transform: translateY(24px); }
  to   { opacity: 1; transform: translateY(0); }
}

.card {
  animation: fadeUp 0.6s ease both;
  transition: transform 0.2s ease, box-shadow 0.2s ease;
}
```

```
.card:hover { transform: translateY(-6px); }
```

## 3. Desenvolvimento — Scripts Python

Os quatro scripts Python constituem o motor matemático do projeto. Todos seguem as mesmas convenções: docstring de módulo no topo, tipagem com anotações (float, int), função main() isolada, tratamento de erros com try/except e comentários em cada bloco lógico.

### 3.1 Custo de Produção — custo\_producao.py

#### Conceito matemático

A função de custo é uma função afim  $C(x) = cf + cv \cdot x$ , onde  $cf$  é o custo fixo (independente da produção),  $cv$  é o custo variável por saca e  $x$  é a quantidade produzida. O ponto de equilíbrio é encontrado igualando receita e custo:  $PV \cdot x = cf + cv \cdot x \rightarrow x^* = cf / (PV - CV)$ .

#### Implementação

```
def custo_total(sacas: float, custo_fixo: float, custo_variavel: float) -> float:
    """C(x) = custo_fixo + custo_variavel * x"""
    # Função afim: parte fixa mais parte proporcional à produção
    return custo_fixo + custo_variavel * sacas

def ponto_equilibrio(custo_fixo: float, preco_venda: float, custo_variavel: float) -> float:
    """Quantidade mínima para lucro zero: x = CF / (PV - CV)"""
    if preco_venda <= custo_variavel:
        raise ValueError("O preço de venda deve ser maior que o custo variável.")
    return custo_fixo / (preco_venda - custo_variavel)
```

```
• (.venv) PS C:\Users\asghr\GitHub\Projeto-AgroCalc-Web> python python/custo_producao.py
Custo fixo (R$): 8000
Custo variável por saca (R$): 35
Preço de venda por saca (R$): 52
Sacas produzidas: 420
Custo total: R$ 22700.00
Ponto de equilíbrio: 470.6 sacas
❖ (.venv) PS C:\Users\asghr\GitHub\Projeto-AgroCalc-Web> []
```

### 3.2 Financiamento Agrícola — financiamento.py

#### Conceito matemático

A fórmula PMT (Payment) calcula a parcela fixa de um financiamento a juros compostos:  $PMT = PV \cdot [i(1+i)^n] / [(1+i)^n - 1]$ , onde  $PV$  é o valor financiado,  $i$  é a taxa mensal e  $n$  é o número de parcelas. A tabela de amortização decompõe cada parcela em juros do período ( $saldo \cdot i$ ) e amortização ( $parcela - juros$ ).

#### Implementação

```
def calcular_parcela(valor_presente: float, taxa: float, parcelas: int) -> float:
    # PMT = PV * [i(1+i)^n] / [(1+i)^n - 1]
```



```

    return valor_presente * (taxa * (1+taxa)**parcelas) / ((1+taxa)**parcelas -
1)

def tabela_amortizacao(valor_presente, taxa, parcelas):
    parcela = calcular_parcela(valor_presente, taxa, parcelas)
    saldo = valor_presente
    for mes in range(1, parcelas + 1):
        juros = saldo * taxa
        amortizacao = parcela - juros
        saldo -= amortizacao

```

```

(.venv) PS C:\Users\asghr\GitHub\Projeto-AgroCalc-Web> python python/financiamento.py
Valor financiado (R$): 50000
Taxa mensal (%): 1.5
Número de parcelas: 24
Parcela fixa: R$ 2496.21

```

| Mês | Parcela     | Juros      | Amort.      | Saldo        |
|-----|-------------|------------|-------------|--------------|
| 1   | R\$ 2496.21 | R\$ 750.00 | R\$ 1746.21 | R\$ 48253.79 |
| 2   | R\$ 2496.21 | R\$ 723.81 | R\$ 1772.40 | R\$ 46481.40 |
| 3   | R\$ 2496.21 | R\$ 697.22 | R\$ 1798.98 | R\$ 44682.41 |
| 4   | R\$ 2496.21 | R\$ 670.24 | R\$ 1825.97 | R\$ 42856.44 |
| 5   | R\$ 2496.21 | R\$ 642.85 | R\$ 1853.36 | R\$ 41003.09 |
| 6   | R\$ 2496.21 | R\$ 615.05 | R\$ 1881.16 | R\$ 39121.93 |
| 7   | R\$ 2496.21 | R\$ 586.83 | R\$ 1909.38 | R\$ 37212.55 |
| 8   | R\$ 2496.21 | R\$ 558.19 | R\$ 1938.02 | R\$ 35274.53 |
| 9   | R\$ 2496.21 | R\$ 529.12 | R\$ 1967.09 | R\$ 33307.45 |
| 10  | R\$ 2496.21 | R\$ 499.61 | R\$ 1996.59 | R\$ 31310.85 |
| 11  | R\$ 2496.21 | R\$ 469.66 | R\$ 2026.54 | R\$ 29284.31 |
| 12  | R\$ 2496.21 | R\$ 439.26 | R\$ 2056.94 | R\$ 27227.37 |
| 13  | R\$ 2496.21 | R\$ 408.41 | R\$ 2087.79 | R\$ 25139.58 |
| 14  | R\$ 2496.21 | R\$ 377.09 | R\$ 2119.11 | R\$ 23020.46 |
| 15  | R\$ 2496.21 | R\$ 345.31 | R\$ 2150.90 | R\$ 20869.57 |
| 16  | R\$ 2496.21 | R\$ 313.04 | R\$ 2183.16 | R\$ 18686.40 |
| 17  | R\$ 2496.21 | R\$ 280.30 | R\$ 2215.91 | R\$ 16470.50 |
| 18  | R\$ 2496.21 | R\$ 247.06 | R\$ 2249.15 | R\$ 14221.35 |
| 19  | R\$ 2496.21 | R\$ 213.32 | R\$ 2282.88 | R\$ 11938.46 |
| 20  | R\$ 2496.21 | R\$ 179.08 | R\$ 2317.13 | R\$ 9621.33  |
| 21  | R\$ 2496.21 | R\$ 144.32 | R\$ 2351.89 | R\$ 7269.45  |
| 22  | R\$ 2496.21 | R\$ 109.04 | R\$ 2387.16 | R\$ 4882.29  |
| 23  | R\$ 2496.21 | R\$ 73.23  | R\$ 2422.97 | R\$ 2459.32  |
| 24  | R\$ 2496.21 | R\$ 36.89  | R\$ 2459.32 | R\$ 0.00     |

```

(.venv) PS C:\Users\asghr\GitHub\Projeto-AgroCalc-Web>

```

### 3.3 Blend de Fertilizante — blend\_fertilizante.py

#### Conceito matemático

O problema de blend é modelado como um sistema linear 3×3:  $A \cdot x = b$ , onde A é a matriz de composição (cada linha representa um nutriente N, P, K; cada coluna um fertilizante), x é o vetor de quantidades em kg/ha e b é o vetor de metas nutricionais. Antes de resolver, calcula-se  $\det(A)$  para garantir solução única ( $\det \neq 0$ ).

#### Implementação

```
import numpy as np

A = np.array([
    [0.45, 0.10, 0.05], # ureia:      45%N   10%P   5%K
    [0.00, 0.46, 0.00], # superfosfato: 0%N   46%P   0%K
    [0.00, 0.00, 0.60], # KCl:         0%N    0%P  60%K
])
b = np.array([90, 60, 60]) # metas: 90kgN, 60kgP, 60kgK

det = np.linalg.det(A) # verifica solução única
if abs(det) > 1e-10:
    x = np.linalg.solve(A, b)
```

```
(.venv) PS C:\Users\asghr\GitHub\Projeto-AgroCalc-Web> python python/blend_fertilizante.py
=== BLENDE DE FERTILIZANTE ===

=== COMPOSIÇÃO DOS FERTILIZANTES ===
(pressione Enter para usar o valor padrão entre colchetes)

Ureia – % nitrogênio [45]: 45
Ureia – % fósforo [10]: 10
Ureia – % potássio [5]: 5
Superfosfato – % fósforo [46]: 46
KCl – % potássio [60]: 60

=== METAS DE NUTRIENTES ===
Meta de nitrogênio (kg/ha) [padrão 90]: 90
Meta de fósforo (kg/ha) [padrão 60]: 60
Meta de potássio (kg/ha) [padrão 60]: 60

Determinante: 0.1242
Ureia:      171.0 kg/ha
Superfosfato: 130.4 kg/ha
KCl:       100.0 kg/ha
(.venv) PS C:\Users\asghr\GitHub\Projeto-AgroCalc-Web>
```

### 3.4 Mapa de Talhões — matriz\_talhoes.py

#### Conceito matemático

A fazenda é representada como uma matriz 3×4 (3 linhas de talhões × 4 setores). NumPy permite operações vetorizadas sobre essa estrutura: média global (mean()), valor máximo (max()), média por linha (mean(axis=1)) e estimativa de produção total a partir de uma área de referência.

#### Implementação

```
import numpy as np

talhoes = np.array([
    [62, 58, 71, 65], # Linha Norte
    [70, 80, 75, 68], # Linha Centro
    [55, 60, 58, 63], # Linha Sul
])

print(f"Média geral: {talhoes.mean():.1f} sacas/ha")
print(f"Melhor setor: {talhoes.max()} sacas/ha")
```

```
print(f"Média por linha: {talhoes.mean(axis=1)}")
print(f"Total (300ha): {talhoes.sum() * 300 / talhoes.size:.0f} sacas")
```

```
• (.venv) PS C:\Users\asghr\GitHub\Projeto-AgroCalc-Web> python python/matriz_talhoes.py
=== MAPA DE PRODUÇÃO ===
Digite a produtividade (sacas/ha) para cada talhão.
Matriz 3 linhas x 4 setores

Linha 1, Setor 1 (padrão 65): 65
Linha 1, Setor 2 (padrão 65): 65
Linha 1, Setor 3 (padrão 65): 56
Linha 1, Setor 4 (padrão 65): 56
Linha 2, Setor 1 (padrão 65): 65
Linha 2, Setor 2 (padrão 65): 75
Linha 2, Setor 3 (padrão 65): 4
Linha 2, Setor 4 (padrão 65): 46
Linha 3, Setor 1 (padrão 65): 7
Linha 3, Setor 2 (padrão 65): 88
Linha 3, Setor 3 (padrão 65): 4
Linha 3, Setor 4 (padrão 65): 33
=== MAPA DE PRODUÇÃO (sacas/ha) ===
[[65. 65. 56. 56.]
 [65. 75.  4. 46.]
 [ 7. 88.  4. 33.]]
Média geral:          47.0 sacas/ha
Melhor setor:         88.0 sacas/ha
Média por linha:      [60.5 47.5 33. ]
Total estimado (300ha): 14100 sacas
❖ (.venv) PS C:\Users\asghr\GitHub\Projeto-AgroCalc-Web> |
```

### 3.5 Servidor Flask e Gráfico Matplotlib — serve\_plot.py

O serve\_plot.py é o bônus de integração Python-web. Ele sobe um servidor Flask na porta 5000 que expõe o endpoint /plot.png. Quando o usuário clica em "Gerar gráfico (Python)" na página calculadora.html, o JavaScript faz uma requisição fetch para /api/generate-plot (roteada pelo server.js para o Flask). O servidor Python gera o gráfico com Matplotlib e retorna a imagem PNG diretamente, sem salvar em disco.

```
from flask import Flask, Response, request
import matplotlib.pyplot as plt

@app.route("/plot.png")
def plot_png():
    values = request.args.get("values")
    # Gera gráfico de barras com cor por desempenho
    # Retorna PNG como Response diretamente (sem disco)
    img = make_plot_from_values(parsed_values, parsed_labels)
    return Response(img, mimetype="image/png")
```

PYTHON

# Calculadoras financeiras no navegador

Os mesmos conceitos dos scripts Python aparecem aqui em formulários rápidos para demonstração ao vivo.

## Mapa de talhões

Matriz 3 linhas x 4 setores. Valores em sacas/ha.

Linha Norte

Setor 1

80

Setor 2

18

Setor 3

23

Setor 4

57

Linha Centro

Setor 1

70

Setor 2

80

Setor 3

75

Setor 4

35

Linha Sul

Setor 1

47

Setor 2

10

Setor 3

62

Setor 4

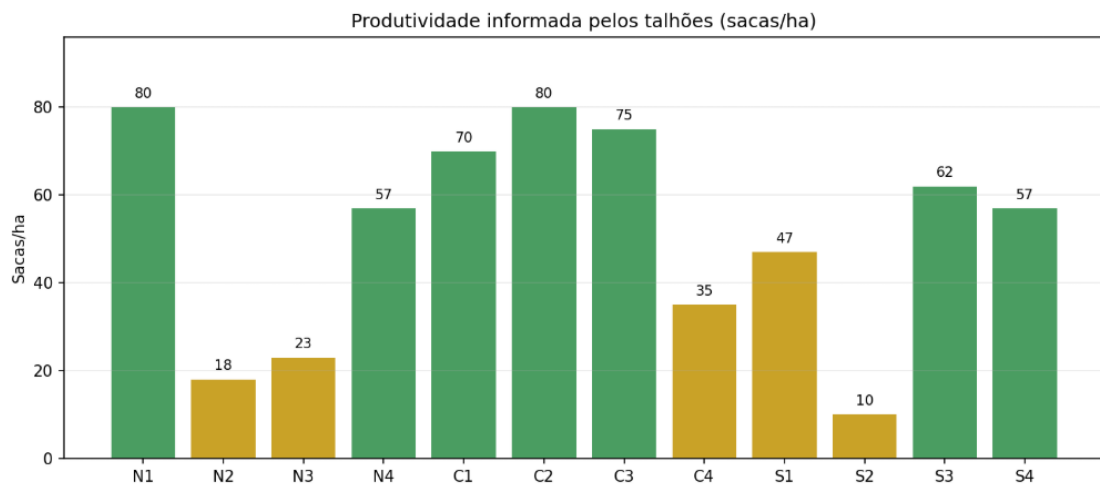
57

Analisar produtividade

Informe os valores para ver a análise de produtividade.

## Gráfico (Matplotlib)

Exemplo de gráfico gerado por um script Python e exibido no site (bônus).



Gerar gráfico (Python)

O gráfico é atualizado pelo servidor Python quando você clica em **Gerar gráfico (Python)**.

## 4. Desenvolvimento — Módulo de Computação

### 4.1 Conversor de Sistemas de Numeração

O conversor, implementado em JavaScript no script.js, recebe um inteiro decimal não-negativo e produz as representações binária e hexadecimal usando os métodos nativos `toString(2)` e `toString(16)`:

```
function updateConverterResult(value, mode) {  
  const dec = Number.parseInt(value, 10);  
  const bin = dec.toString(2);          // base 2  
  const hex = dec.toString(16).toUpperCase(); // base 16  
  // ex: 175 → binário: 10101111 | hex: AF  
}
```

| Decimal | Binário  | Hexadecimal | Contexto                      |
|---------|----------|-------------|-------------------------------|
| 175     | 10101111 | AF          | Código de sensor de umidade   |
| 255     | 11111111 | FF          | Valor máximo de um byte       |
| 16      | 10000    | 10          | Base da numeração hexadecimal |

MÓDULOS PRÁTICOS

Ferramentas de apoio à decisão

Use os formulários abaixo para testar a lógica do projeto no navegador.

Conversor numérico

Valor decimal

199

Base de saída

Binário e hexadecimal

Converter

Binário: 11000111  
Hexadecimal: C7  
Valor decimal: 199

### 4.2 Simulador de Porta Lógica AND — Decisão de Irrigação

A porta AND é implementada no JavaScript e exibida em tempo real na página `calculadora.html`. O princípio: irrigar somente quando umidade E temperatura estiverem dentro do limite aceitável (ambas as entradas = 1). Um único 0 na entrada produz 0 na saída.

| Umidade | Temperatura | AND (Irigar?) |
|---------|-------------|---------------|
|---------|-------------|---------------|

|   |   |                 |
|---|---|-----------------|
| 0 | 0 | 0 — Não irrigar |
| 0 | 1 | 0 — Não irrigar |
| 1 | 0 | 0 — Não irrigar |
| 1 | 1 | 1 — Irrigar     |

A tabela verdade estática está também em `computacao.html`, enquanto o formulário em `calculadora.html` avalia dinamicamente o par escolhido pelo usuário.

### Decisão de irrigação

Umidade dentro do limite?

Sim

Temperatura dentro do limite?

Sim

Avaliar

**Tabela avaliada:**

Umidade = 1

Temperatura = 1

AND = 1

Decisão: irrigar

## 4.3 Página "Como Funciona" — Arquitetura de Computadores

A página `computacao.html` explica, em três cards, como CPU, RAM e Armazenamento participam da execução do AgroCalc Web:

| Componente    | Papel no AgroCalc Web                                               |
|---------------|---------------------------------------------------------------------|
| CPU           | Processa as instruções do navegador e dos scripts Python            |
| RAM           | Guarda dados temporários durante a execução e exibição dos cálculos |
| Armazenamento | Contém os arquivos HTML, CSS, JavaScript e Python do projeto        |

## 4.4 Linha do Tempo da Computação no Agronegócio

Cinco marcos históricos são apresentados em layout de timeline CSS (grid com estilização por `article.timeline-item`):

| Ano  | Marco                                                              |
|------|--------------------------------------------------------------------|
| 1950 | Primeiros registros mecanizados de produção e máquinas de calcular |
| 1970 | Planilhas e computadores começam a apoiar planejamento rural       |

|       |                                                                  |
|-------|------------------------------------------------------------------|
| 1990  | Sistemas de gestão agrícola organizam dados de lavoura e estoque |
| 2010  | Internet no campo e sensores conectam a agricultura              |
| 2020+ | Aplicações web e análise de dados em tempo real                  |

### 4.5 Teoria dos Conjuntos Aplicada a Insumos Agrícolas

A página `computacao.html` demonstra a teoria de conjuntos classificando insumos agrícolas em três grupos:

| Conjunto      | Critério                              | Relação                            |
|---------------|---------------------------------------|------------------------------------|
| Orgânicos (O) | Origem biológica, sem síntese química | $O \cap Q = \emptyset$ (disjuntos) |
| Químicos (Q)  | Sintetizados industrialmente          | $Q \cap O = \emptyset$ (disjuntos) |
| Híbridos (H)  | Combinação orgânico + mineral         | $H \subset O \cup Q$               |

## 5. Dificuldades e Aprendizados

---

### 5.1 Integração Python-Web

O maior desafio técnico foi fazer o gráfico Matplotlib aparecer no navegador em tempo real. A abordagem inicial de gerar um arquivo PNG estático e referenciá-lo no HTML funcionava, mas não permitia atualização dinâmica. A solução adotada foi criar um servidor Flask que gera a imagem em memória (BytesIO) e a retorna como resposta HTTP, sem jamais gravar em disco — o que exigiu compreender o ciclo de vida de requisição HTTP e o uso da API fetch no JavaScript.

### 5.2 Dark Mode e Persistência de Estado

Implementar o toggle de tema que persiste entre navegações exigiu entender como o localStorage funciona e seus limites (modo privado não permite escrita). O tratamento com try/catch garante que o tema padrão dark seja aplicado mesmo quando o localStorage está bloqueado.

### 5.3 Sistema Linear no Navegador sem NumPy

Os scripts Python usam `numpy.linalg.solve()`, que abstrai todo o processo de resolução. No JavaScript (para o formulário de blend na calculadora.html) foi necessário implementar manualmente o método da matriz adjunta: calcular o determinante pela regra de Sarrus, montar a matriz de cofatores, transpor para obter a adjunta e multiplicar pelo vetor  $b$ . Esse exercício aprofundou o entendimento do que NumPy faz internamente.

### 5.4 Responsividade em Formulários com Muitos Campos

A grade de talhões (12 inputs em layout 3×4) apresentou problemas de legibilidade em telas pequenas. A solução foi usar a classe `.talhoes-row` com `display: grid` e `grid-template-columns` responsivo, além de labels abreviados em mobile via media query.



## 6. Conclusão

---

O AgroCalc Web atende aos seis módulos previstos no projeto: Painel Principal, Calculadora de Custo, Financiamento Agrícola, Conversor de Dados, Mapa de Talhões e Blend de Fertilizante. Todos os requisitos técnicos obrigatórios foram implementados — HTML semântico, CSS externo com variáveis, responsividade, animações, scripts Python documentados com try/except, conversor de numeração, tabela verdade AND e linha do tempo. O bônus de dark mode e o bônus de gráfico Matplotlib integrado ao site foram igualmente entregues.

O projeto demonstrou, na prática, que as três disciplinas do semestre não são áreas isoladas: a função afim de Matemática Aplicada aparece tanto no código Python quanto no formulário HTML; o conceito de sistema linear conecta NumPy no terminal e a matriz adjunta no JavaScript; a conversão de bases une a teoria de Computação ao contexto real de sensores IoT no campo.

Como possibilidade de expansão, o sistema poderia incorporar: autenticação por produtor para salvar histórico de cálculos, integração com APIs de cotação de commodities para atualizar automaticamente o preço de venda das sacas, e exportação dos resultados em PDF direto do navegador.

## 7. Referências

---

- MDN Web Docs. HTML: HyperText Markup Language. Disponível em: <https://developer.mozilla.org/pt-BR/docs/Web/HTML>. Acesso em: mai. 2025.
- MDN Web Docs. CSS: Cascading Style Sheets. Disponível em: <https://developer.mozilla.org/pt-BR/docs/Web/CSS>. Acesso em: mai. 2025.
- Python Software Foundation. Python 3 Documentation. Disponível em: <https://docs.python.org/3/>. Acesso em: mai. 2025.
- NumPy Developers. NumPy Reference. Disponível em: <https://numpy.org/doc/stable/reference/>. Acesso em: mai. 2025.
- Matplotlib Development Team. Matplotlib Documentation. Disponível em: <https://matplotlib.org/stable/contents.html>. Acesso em: mai. 2025.
- Flask Project. Flask Documentation. Disponível em: <https://flask.palletsprojects.com/>. Acesso em: mai. 2025.
- Google Fonts. Inter typeface. Disponível em: <https://fonts.google.com/specimen/Inter>. Acesso em: mai. 2025.
- LAPPONI, Jurandir Sell. Pesquisa Operacional. São Paulo: Lapponi Treinamento e Editora, 2004.
- STEWART, James. Cálculo. Vol. 1. São Paulo: Cengage Learning, 2013.